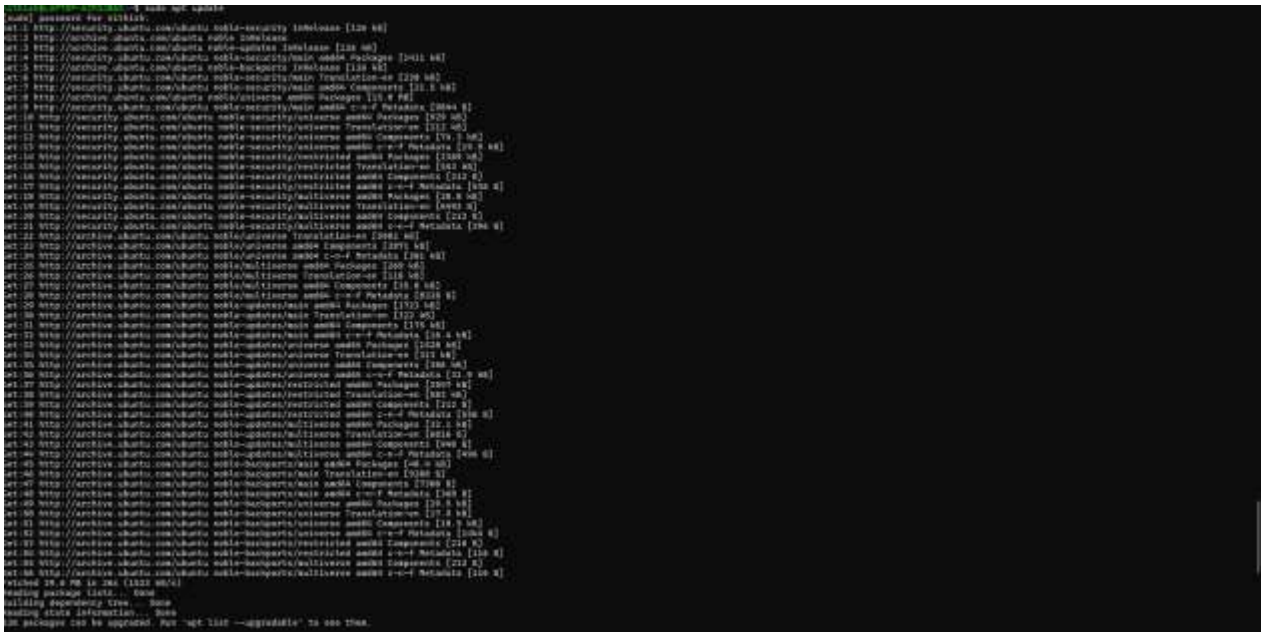
HIGH PERFORMANCE COMPUTING Lab Ass-4

Name: SUJAL

Batch-11

Hall Ticket:2303A51713

Lab: OpenMP Parallel Loops (From Serial to Parallel)
Task 1: OpenMP Parallel Loops (From Serial to Parallel)
Objective: Confirm OpenMP-ready system.
Steps:
Lscpu
nproc
gcc –version
Expected Observation:
Multiple CPU cores available
GCC installed with OpenMP support



Task 2: Serial Baseline – Loop Computation
Objective: Establish serial execution time.
File: sum_serial.c
#include <stdio.h>
#include <time.h>

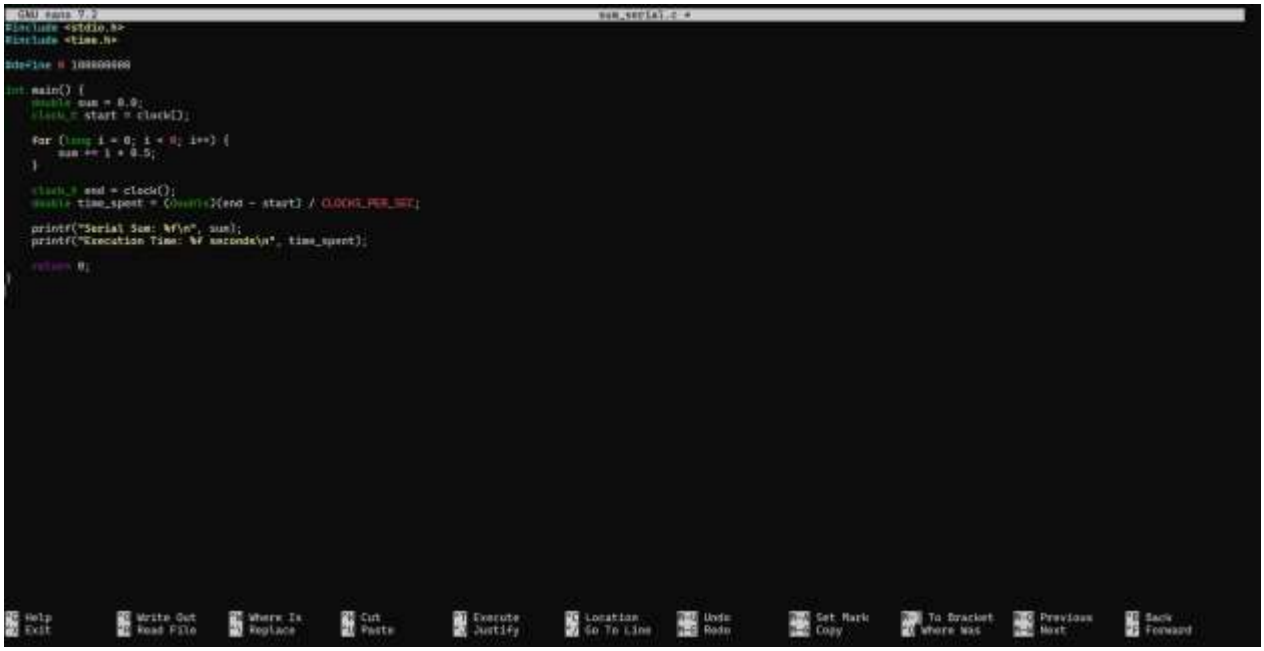
04-02-2026 EOD

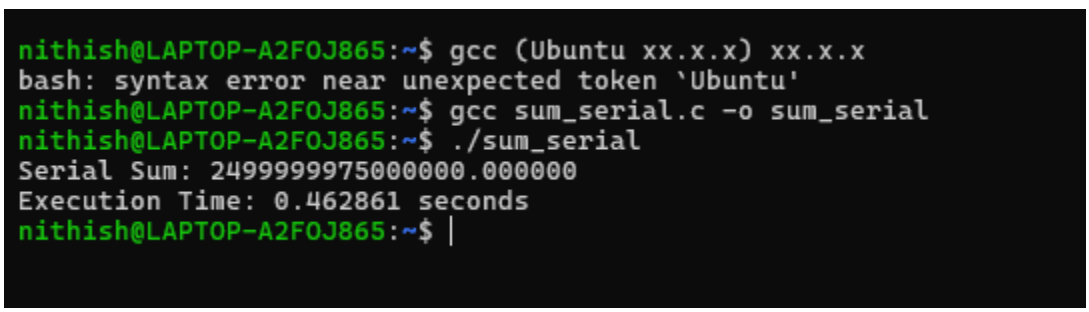
```
#define N 100000000

int main() {
double sum = 0.0;
clock_t start = clock();

for (long i = 0; i &lt; N; i++) {
sum += i * 0.5;
}

clock_t end = clock();
double time_spent = (double)(end - start) /
CLOCKS_PER_SEC;

printf(&quot;Serial Sum: %f\n&quot;, sum);
printf(&quot;Execution Time: %f seconds\n&quot;, time_spent);
return 0;
}
```

Compile & Run:

```
&gt; gcc sum_serial.c -o sum_serial
&gt; ./sum_serial
```

Submission: Screenshot showing execution time.





Task 3: OpenMP Parallel Loop
Objective: Parallelize the loop using OpenMP.
File: sum_openmp.c

04-02-2026 EOD

```
#include &lt;stdio.h&gt;
#include &lt;omp.h&gt;

#define N 100000000

int main() {
double sum = 0.0;
double start = omp_get_wtime();

#pragma omp parallel for reduction(+:sum)
for (long i = 0; i &lt; N; i++) {
sum += i * 0.5;
```

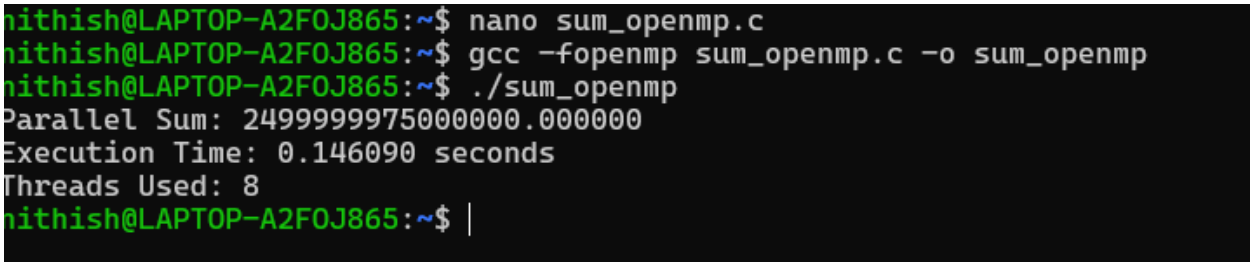
}

double end = omp_get_wtime();

printf("Parallel Sum: %f\n", sum);
printf("Execution Time: %f seconds\n", end - start);
printf("Threads Used: %d\n", omp_get_max_threads());

return 0;
}
Compile:

> gcc -fopenmp sum_openmp.c -o sum_openmp



Task 4: Thread Scaling Experiment
Run with different thread counts:
export OMP_NUM_THREADS=1
./sum_openmp

export OMP_NUM_THREADS=2
./sum_openmp

export OMP_NUM_THREADS=4
./sum_openmp

export OMP_NUM_THREADS=8
./sum_openmp

Record the readings in a below table:

```
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=1
./sum_openmp
Parallel Sum: 2499999975000000.000000
Execution Time: 0.551355 seconds
Threads Used: 1
nithish@LAPTOP-A2FOJ865:~$
```

```
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=2
./sum_openmp
Parallel Sum: 2499999975000000.000000
Execution Time: 0.236198 seconds
Threads Used: 2
nithish@LAPTOP-A2FOJ865:~$
```

```
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=4
./sum_openmp
Parallel Sum: 2499999975000000.000000
Execution Time: 0.143396 seconds
Threads Used: 4
nithish@LAPTOP-A2FOJ865:~$
```

```
nithish@LAPTOP-A2FOJ865:~$ export OMP_NUM_THREADS=8
./sum_openmp
Parallel Sum: 2499999975000000.000000
Execution Time: 0.117494 seconds
Threads Used: 8
nithish@LAPTOP-A2FOJ865:~$
```

Threads	Execution Time (s)
1	0.551355 seconds
2	0.236198 seconds

4	0.143396 seconds
8	0.117494 seconds

Task 5:
Speedup

S(p)=T1/Tp
Efficiency
E(p)=S(p)/p
Compute for 2, 4, 8 threads.
Submission: Table with Speedup & Efficiency.

Step 1: Identify execution times

From **Task 4**, you already have:

- **T1** → execution time with **1 thread**
- **T2** → execution time with **2 threads**
- **T4** → execution time with **4 threads**
- **T8** → execution time with **8 threads**

Step 2: Use formulas

Speedup:
S(p)=T1TpS(p) = \frac{T_1}{T_p}S(p)=TpT1

Efficiency:

E(p)=S(p)pE(p) = \frac{S(p)}{p}E(p)=pS(p)

Step 3: Example calculation

Assume (example values):

- T1 = 1.20 s
- T2 = 0.65 s
- T4 = 0.38 s
- T8 = 0.30 s

For 2 threads:

- S(2) = 1.20 / 0.65 = **1.84**
- E(2) = 1.84 / 2 = **0.92**

For 4 threads:

- S(4) = 1.20 / 0.38 = **3.15**
- E(4) = 3.15 / 4 = **0.79**

For 8 threads:

- S(8) = 1.20 / 0.30 = **4.00**
- E(8) = 4.00 / 8 = **0.50**

Threads (p)	Time Tp (s)	Speedup S(p)	Efficiency E(p)
2	T2	T1 / T2	S(2) / 2
4	T4	T1 / T4	S(4) / 4
8	T8	T1 / T8	S(8) / 8

1. Why does OpenMP work well for this loop?

OpenMP works well here because every loop iteration is independent. Each thread can calculate its part without waiting for others, and at the end OpenMP safely adds all the partial results together using the reduction clause.

2. What happens if loop iterations are dependent?

If the iterations depend on each other, running them in parallel can give wrong results. This happens because multiple threads may try to read or update the same data at the same time, causing race conditions. Adding synchronization can fix this, but it also slows down the program.

3. Why does speedup stop improving after some threads?

After a certain number of threads, performance stops improving because the system has limited CPU cores. Also, managing many threads takes extra time, and they start competing for memory. Some parts of the program still run serially, which limits overall speedup.

4. Is OpenMP suitable for multi-node systems? Why or why not?

No, OpenMP is mainly meant for systems where all threads share the same memory, like a single machine. In multi-node systems, each node has its own memory, so communication is needed. For that, MPI is more suitable than OpenMP.